PAYMENT ENTITY – SQL DATABASE REPORT

-HEMAVATHI.B (E25AI114)

SQL Implementation, Constraints and Payment Transaction Analysis

1. Objective

To create and manage a Payment Entity using SQL by implementing table creation, constraints, payment records, and queries for successful, failed and pending transactions.

2. Payment Entity

Attribute	Description
Payment_ID	Unique payment identification
Order_ID	Related order identification
Payment_Date	Date of payment
Payment_Mode	UPI, Card, Cash, etc.
Payment_Status	Success, Failed or Pending
Transaction_Amount	Amount paid

3. Create Payment Table

```
SQL> CREATE TABLE Payment(PAYMENT_ID VARCHAR2(10),ORDER_ID VARCHAR2(7),PAYMENT_DATE DATE,PAYMENT_MODE VARCHAR2(20),PAYMENT_STATUS VARCHAR2(20),TRANS
ACTION_AMOUNT NUMBER(10,2));

Table created.
```

4. Constraints

Primary Key

ALTER TABLE Payment ADD PRIMARY KEY (Payment_ID);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT PK_PAYMENT PRIMARY KEY(PAYMENT_ID);

Table altered.
```

Foreign Key

ALTER TABLE Payment ADD CONSTRAINT fk_order FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT FK_PAYMENT_ORDER FOREIGN KEY(ORDER_ID) REFERENCES Orders(ORDER_ID);

Table altered.
```

Check Constraint

ALTER TABLE Payment ADD CONSTRAINT chk_amount CHECK (Transaction_Amount > 0);

```
SQL> ALTER TABLE Payment ADD CONSTRAINT CHK_PAYMENT_AMOUNT CHECK(TRANSACTION_AMOUNT>0);

Table altered.
```

Not NULL Constraint

ALTER TABLE Payment MODIFY(PAYMENT_MODE NOT NULL,PAYMENT_STATUS NOT NULL);

```
SQL> ALTER TABLE Payment MODIFY(PAYMENT_MODE NOT NULL,PAYMENT_STATUS NOT NULL);

Table altered.
```

5. Insert Payment Records

```
SQL> INSERT INTO Payment VALUES('PAY001','ORD001',TO_DATE('25-SEP-2026','DD-MON-YYYY'),'UPI','Success',2500);

1 row created.

SQL> INSERT INTO Payment VALUES('PAY002','ORD002',TO_DATE('25-SEP-2026','DD-MON-YYYY'),'Card','Pending',1800);

1 row created.

SQL> INSERT INTO Payment VALUES('PAY003','ORD003',TO_DATE('24-SEP-2026','DD-MON-YYYY'),'Net Banking','Success',3500);

1 row created.

SQL> INSERT INTO Payment VALUES('PAY004','ORD004',TO_DATE('23-SEP-2026','DD-MON-YYYY'),'Cash','Success',1200);

1 row created.

SQL> INSERT INTO Payment VALUES('PAY005','ORD005',TO_DATE('22-SEP-2026','DD-MON-YYYY'),'UPI','Failed',4500);

1 row created.
```

Insert Payment Records

```
SQL> SELECT * FROM Payment;

PAYMENT_ID ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
PAY001     ORD001  25-SEP-26 UPI                  Success
              2500

PAY002     ORD002  25-SEP-26 Card                 Pending
              1800

PAY003     ORD003  24-SEP-26 Net Banking          Success
              3500


PAYMENT_ID ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
PAY004     ORD004  23-SEP-26 Cash                 Success
              1200

PAY005     ORD005  22-SEP-26 UPI                  Failed
              4500
```

6. Display All Successful Payments

SELECT * FROM Payment WHERE Payment_Status = 'Success';

```
SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS='Success';

PAYMENT_ID ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
PAY001     ORD001  25-SEP-26 UPI                  Success
              2500

PAY003     ORD003  24-SEP-26 Net Banking          Success
              3500

PAY004     ORD004  23-SEP-26 Cash                 Success
              1200
```

7. Find All Failed Transactions

SELECT * FROM Payment WHERE Payment_Status = 'Failed';

```
SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS='Failed';

PAYMENT_ID ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
PAY005     ORD005  22-SEP-26 UPI                  Failed
              4500
```

8. Count Total Successful and Failed Payments

SELECT Payment_Status, COUNT(*) AS Total_Payments FROM Payment WHERE Payment_Status IN ('Success', 'Failed') GROUP BY Payment_Status;

```
SQL> SELECT PAYMENT_STATUS,COUNT(*) AS TOTAL_PAYMENTS FROM Payment WHERE PAYMENT_STATUS IN ('Success','Failed') GROUP BY PAYMENT_STATUS;

PAYMENT_STATUS       TOTAL_PAYMENTS
-------------------- --------------
Failed                            1
Success                           3
```

9. Update Failed Payments After Retry

```
SQL> UPDATE Payment SET PAYMENT_STATUS='Success' WHERE PAYMENT_ID='PAY005';

1 row updated.
```

10. Identify Pending Transactions

SELECT * FROM Payment WHERE Payment_Status = 'Pending';

```
SQL> SELECT * FROM Payment WHERE PAYMENT_STATUS='Pending';

PAYMENT_ID ORDER_I PAYMENT_D PAYMENT_MODE         PAYMENT_STATUS
---------- ------- --------- -------------------- --------------------
TRANSACTION_AMOUNT
------------------
PAY002     ORD002  25-SEP-26 Card                 Pending
              1800
```

REPORT-1 (Payment Mode Analysis)

Find:

- Number of transactions using UPI.
- Number of card payments.
- Most preferred payment method.

```
SQL>  SELECT COUNT(*) AS UPI_TRANSACTIONS FROM PAYMENT WHERE PAYMENT_MODE ='UPI';

UPI_TRANSACTIONS
----------------
               2
```

```
SQL> SELECT COUNT(*) AS CARD_PAYMENTS FROM PAYMENT WHERE PAYMENT_MODE ='Card';

CARD_PAYMENTS
-------------
            2
```

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE ORDER BY TRANSACTION_COUNT DESC;

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- ----------------
Card                                2
UPI                                 2
Cash                                1
```

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM payment GROUP BY PAYMEN
T_MODE);

PAYMENT_MODE         TRANSACTION_COUNT
-------------------- ----------------
Card                                2
UPI                                 2
```

Report 2: Revenue Analysis

Find:

- Total revenue generated.
- Revenue by payment method.
- Average transaction amount.

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT FROM PAYMENT GROUP BY PAYMENT_MODE HAVING COUNT(*)=(SELECT MAX(COUNT(*)) FROM payment GROUP BY PAYMEN
T_MODE);

PAYMENT_MODE       TRANSACTION_COUNT
------------------ -----------------
Card                               2
UPI                                2
```

```
SQL> SELECT PAYMENT_MODE, SUM(TRANSACTION_AMOUNT) AS REVENUE
  2  FROM PAYMENT
  3  WHERE UPPER(PAYMENT_STATUS) = 'SUCCESS'
  4  GROUP BY PAYMENT_MODE
  5  ORDER BY REVENUE DESC;

PAYMENT_MODE             REVENUE
-------------------- -----------
Card                        4550
UPI                         4000
```

```
SQL> SELECT ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVERAGE_TRANSACTION_AMOUNT FROM PAYMENT WHERE PAYMENT_STATUS='Success';

AVERAGE_TRANSACTION_AMOUNT
--------------------------
                    2137.5
```

REPORT-3 Customer Payment History

Display:

- Customer name
- Order ID
- Payment mode
- Amount
- Payment status

```
SQL> SELECT C.CUSTOMER_NAME,O.ORDER_ID,O.ORDER_DATE,P.PAYMENT_DATE,P.PAYMENT_MODE,P.TRANSACTION_AMOUNT AS AMOUNT,P.PAYMENT_STATUS FROM CUSTOMER C JOIN ORDER
S O ON C.CUSTOMER_ID=O.CUSTOMER_ID JOIN PAYMENT P ON O.ORDER_ID=P.ORDER_ID ORDER BY P.PAYMENT_DATE;

CUSTOMER_NAME                  ORDER_I ORDER_DAT PAYMENT_D PAYMENT_MODE
------------------------------ ------- --------- --------- --------------------
    AMOUNT PAYMENT_STATUS
---------- --------------------
Devi                           0101    01-SEP-26 20-SEP-26 UPI
      2500 Success

Radhi                          0102    02-SEP-26 21-SEP-26 Card
      1800 Success

Ajay                           0103    03-SEP-26 22-SEP-26 Cash
      3200 Pending


CUSTOMER_NAME                  ORDER_I ORDER_DAT PAYMENT_D PAYMENT_MODE
------------------------------ ------- --------- --------- --------------------
    AMOUNT PAYMENT_STATUS
---------- --------------------
Vijay                          0104    04-SEP-26 23-SEP-26 UPI
      1500 Success

Abi                            0105    05-SEP-26 24-SEP-26 Card
      2750 Success
```

Payment Analysis:

```
SQL> SELECT PAYMENT_MODE,COUNT(*) AS TRANSACTION_COUNT,SUM(TRANSACTION_AMOUNT) AS TOTAL_REVENUE,ROUND(AVG(TRANSACTION_AMOUNT),2) AS AVG_TRANSACTION FROM PAY
MENT WHERE PAYMENT_STATUS='Success' GROUP BY PAYMENT_MODE ORDER BY TOTAL_REVENUE DESC;

PAYMENT_MODE         TRANSACTION_COUNT TOTAL_REVENUE AVG_TRANSACTION
-------------------- ----------------- ------------- ---------------
Card                                 2          4550            2275
UPI                                  2          4000            2000
```

11. Conclusion

The Payment Entity was successfully created and implemented using SQL. Primary key, foreign key and check constraints were applied to maintain data integrity. Payment records were inserted and analyzed using SQL